Manual de administrador

Instalación, despliegue, arquitectura y mantenimiento

«Vive la aventura de descubrir el poder de tus emociones»

Proyecto: EMO-AVENTURA · Isla Emociones 3D
Repositorio: github.com/Michaelpob/Emo-aventura
Sitio: michaelpob.github.io/Emo-aventura/
Versión del documento: 1.0
Fecha: Septiembre de 2026

Tabla de contenido

1. Resumen técnico
2. Requisitos
   - Para jugar
   - Para desarrollar
3. Instalación y ejecución
4. Compilación y despliegue
   - Compilación
   - Compilación sin Node
   - Despliegue
5. Estructura del proyecto
6. Arquitectura del código
   - Flujo de la aplicación
   - Contrato de un minijuego
   - Islas con varios niveles
7. Catálogo de minijuegos registrados
8. Datos: qué se guarda y dónde
   - Estadísticas anónimas de uso (Supabase)
9. Editar contenido sin tocar la lógica
10. Añadir una isla o un minijuego
11. Audio: cómo funciona y cómo añadir sonidos
12. Rendimiento y accesibilidad en el código
13. Pruebas y verificación
    - Comprobación rápida
    - Pruebas de extremo a extremo (recomendado)
14. Control de versiones y flujo de trabajo
15. Resolución de problemas
16. Mantenimiento y hoja de ruta

1. Resumen técnico

Concepto	Valor
Tipo de aplicación	Web estática de una sola página (SPA), 100 % cliente
Lenguaje	JavaScript ES2022 (módulos ES), sin framework
Motor 3D	three.js ^0.179
Empaquetador	Vite ^7.1
Gestor de paquetes	pnpm (vía corepack)
Audio	Web Audio API — **todo el sonido se sintetiza en el navegador**, sin archivos
Persistencia	`localStorage` del navegador: el juego no necesita cuentas ni servidor
Estadísticas	Opcionales: eventos anónimos en Supabase y panel propio en `admin.html`
Publicación	GitHub Pages desde la carpeta `/docs` de la rama `main`
URL pública	https://michaelpob.github.io/Emo-aventura/
Dependencias en tiempo de ejecución	Solo three.js (se empaqueta)

No hay backend, base de datos ni API: **el proyecto se despliega copiando ficheros estáticos**.

---

2. Requisitos

Para jugar

- Navegador con **WebGL 2** y **Web Audio**: Chrome/Edge 100+, Firefox 100+, Safari 15+.
- Cualquier equipo con GPU integrada moderna. El juego ajusta la resolución interna sobre la marcha para sostener la fluidez.

Para desarrollar

- **Node.js 18 o superior** (probado con Node 24).
- **pnpm**, disponible a través de corepack:

```
corepack pnpm --version
```

- Opcional: **Playwright** (ya en `devDependencies`) para las pruebas automatizadas, y un navegador Chromium/Edge instalado.
- Opcional: **Python 3** solo si hay que compilar en una máquina sin Node (ver §4).

---

3. Instalación y ejecución

```
git clone https://github.com/Michaelpob/Emo-aventura.git
cd Emo-aventura
corepack pnpm install
```

Comando	Qué hace
`corepack pnpm dev`	Servidor de desarrollo con recarga en caliente (http://localhost:5173)
`corepack pnpm build`	Compila la versión de producción en `/docs`
`corepack pnpm preview`	Compila y sirve el resultado para revisarlo
`corepack pnpm fps`	Mide fotogramas por segundo (`scripts/measure-fps.mjs`)

> **Nota operativa:** en el equipo de desarrollo habitual el servidor de desarrollo consume demasiados recursos. La forma recomendada de verificar es `build` + **Playwright sin cabeza** (ver §13), que no deja ningún servidor abierto.

---

4. Compilación y despliegue

Compilación

```
corepack pnpm build
```

- Salida: **`/docs`** (`outDir` configurado en `vite.config.js`), que es la carpeta que publica GitHub Pages.
- `base: './'` — las rutas son relativas, así que el sitio funciona igual en la raíz del dominio o en un subdirectorio.
- `three` se separa en su propio *chunk* (`manualChunks`) para que se cachee entre versiones.
- Cada compilación genera un **sello de versión** (`__BUILD__`, también escrito en `docs/version.json`). La página compara ambos al arrancar y **se recarga sola** si el navegador le sirvió un `index.html` viejo de caché.
- El *plugin* `copy-docs` copia a `/docs` los documentos sueltos declarados en `vite.config.js` (`PLAN-GAMEPLAY.md`, `FEAR-LEVEL2.md`).

> Como el sello cambia en cada compilación, **`/docs` siempre aparece modificado** aunque no cambie el código. Si compilas solo para probar, revierte la carpeta (`git checkout -- docs`) antes de commitear.

Compilación sin Node

```
python scripts/build-docs.py
```

Genera un `/docs` equivalente sin minificar, usando *importmap* y copiando three.js desde `node_modules`. Es un plan B; la salida de Vite es la oficial.

Despliegue

1. Compila (`corepack pnpm build`).
2. Commitea `/docs` junto con los cambios de `src/`.
3. `git push origin main`.
4. GitHub Pages publica automáticamente en 1–3 minutos.

Configuración de Pages: **rama `main`, carpeta `/docs`**. No hay acción de CI: el artefacto se versiona en el repositorio.

> El remoto local puede seguir llamándose `isla-emociones-3d`; funciona por la redirección 301 de GitHub tras el renombrado a `Emo-aventura`.

---

5. Estructura del proyecto

```
Emo-aventura/
 index.html                  punto de entrada del juego
 admin.html                  punto de entrada del panel (pagina aparte)
 .env.example                plantilla de variables (Supabase); el .env no se sube
 vite.config.js              build -> /docs, sello de version, copia de documentos
 package.json                scripts y dependencias
 docs/                       SALIDA compilada · lo que publica GitHub Pages
 public/
    valle-luz/index.html    juego autocontenido (Alegria)
    valle-bruma/index.html  juego autocontenido (Tristeza)
 manuales/                   ESTA documentacion
    MANUAL-DE-USUARIO.md
    MANUAL-DE-ADMINISTRADOR.md
    REFERENCIAS-Y-NORMAS.md
    PANEL-ADMIN-CONFIGURACION.md
 supabase/
    schema.sql              tablas, permisos y funciones del panel
 scripts/
    build-docs.py           build alternativo sin Node
    measure-fps.mjs         medicion de rendimiento
    serve-dist.mjs          servidor estatico para `preview`
    verify.mjs              comprobaciones automaticas
 src/
     main.js                 arranque: carga estilos y monta la app
     engine/                 nucleo 3D compartido
        MinigameBase.js     ciclo de vida, HUD, pausa, portal, reset
        PlayerController.js WASD, puntero, 3.ª persona, gravedad, colisiones
        Interactable.js     radio de activacion, chip [E], realce
        Feedback.js         particulas con pooling, flash, shake, tweens
        AudioBus.js         sonidos sintetizados, audio posicional, ducking
        CalmMusic.js        musica generativa de la Calma
        SadnessMusic.js     musica generativa de la Tristeza
        MysteryMusic.js     musica de la casa del Miedo
        BreathPause.js      respiracion guiada reutilizable
        worldkit.js         terreno, cielo, luces, avatar, InstancedMesh
        Stage.js            motor 2D (tarjetas psicoeducativas)
        activities.js       actividades 2D reutilizables
     minigames/
        index.js            REGISTRO de minijuegos
        anger/ fear/ sadness/ joy/ frustration/ calm/ surprise/
        disgust/            isla del Desagrado (ver detalle abajo)
     data/
        islands.js          definicion de las seis islas del mapa
```

```
    gameState.js        progreso, puntos, herramientas, reevaluaciones
    tools.js            catalogo de herramientas e insignias
    player.js           perfil del jugador
  three/WorldScene.js    mapa 3D principal (hub)
  ui/                    EmotionIslandApp.js (navegacion) y screens.js
  services/              supabaseClient.js (carga perezosa) y tracking.js
  admin/                 el panel: main.js, auth.js, dashboard.js, charts.js
  styles/                island3d.css + hojas por isla
```

Detalle de `src/minigames/disgust/` (patrón recomendado para islas nuevas):

Archivo	Responsabilidad
`DisgustIslandFlow.js`	Menú de la isla; lanza los dos niveles y cierra la isla
`DisgustCavesGame.js`	Nivel 1 · Las cuatro cuevas
`DisgustMirrorGame.js`	Nivel 2 · El espejo de las señales
`DesagradoTextos.js`	**Todo el contenido editable** (cuevas, elementos, señales, escenas, textos)
`desagradoAssets.js`	Geometrías y texturas generadas por código (marcadas `PLACEHOLDER_*`)
`desagradoSonidos.js`	Sonidos sintetizados de la caverna de los sonidos
`desagradoStore.js`	Persistencia de respuestas (`emoaventura.desagrado`)
`desagradoUi.js`	Guía, anclas DOM3D, encuadre responsivo

---

6. Arquitectura del código

Flujo de la aplicación

```
index.html  src/main.js  EmotionIslandApp




   pantalla inicio         WorldScene (mapa 3D)     minigameRegistry
   perfil / progreso       seis islas tocables       clase del minijuego
```

`EmotionIslandApp` (en `src/ui/`) es el único que conoce las pantallas: inicio perfil mapa ficha de isla minijuego resultado mapa. Expone la instancia en `window.emoAventura` (útil para depurar y para las pruebas automatizadas).

Contrato de un minijuego

Toda isla implementa el mismo contrato, lo implemente una clase 3D o un «flujo» de varios niveles:

```
new Game({ host, island, player, onComplete, onExit });
game.mount();     // construye y arranca
game.dispose();   // libera TODO (geometrías, materiales, listeners, audio)
```

`MinigameBase` aporta: bucle con `step(dt)` (también para pruebas deterministas), HUD (`showIntro`, `showChoice`, `say`, `showNote`, `addBar`, `setObjective`, `advanceObjective`, `showClosingCard`), pausa con menú, reinicio sin recargar, portal de salida, escalado dinámico de resolución, y limpieza completa en `dispose()`.

Métodos que implementa cada isla: `build()`, `onStart()`, `onUpdate(dt)`, `onReset()`, `onDispose()` y la propiedad `completionPayload`.

Islas con varios niveles

Las islas de Miedo, Frustración y Desagrado usan un **flujo** (`*IslandFlow.js`) que cumple el mismo contrato y lanza cada nivel por dentro, garantizando que **nunca haya dos niveles vivos a la vez**.

---

7. Catálogo de minijuegos registrados

`src/minigames/index.js` mapea identificadores clases. Los que aparecen en el mapa se declaran en `src/data/islands.js` (campo `minigame`).

Identificador	Clase	Estado
`fear-night-house`	`FearIslandFlow`	**En el mapa** (Miedo: bosque + casa)
`anger-lava`	`AngerLavaGame`	**En el mapa** (Enojo)
`disgust-cuevas`	`DisgustIslandFlow`	**En el mapa** (Desagrado: cuevas + espejo)
`sadness-house`	`SadnessHouseGame`	**En el mapa** (Tristeza)
`frustration-tower-valley`	`FrustrationIslandFlow`	**En el mapa** (Frustración: torre + valle)
`calm-lake`	`CalmLakeGame`	**En el mapa** (Calma)
`fear-night`	`FearNightGame`	Alternativo: solo el bosque
`frustration-tower`	`FrustrationTowerGame`	Alternativo: solo la torre
`sadness-days`	`SadnessDaysGame`	Alternativo: «Un día a la vez» (economía por turnos)
`sadness-restore`	`SadnessRestoreGame`	Alternativo: «El mundo que vuelve»
`joy-orbs`	`JoyOrbsGame`	Nivel 2 de Frustración (Valle de la Luz 3D)
`surprise-observe`	`SurpriseObserveGame`	Isla de la Sorpresa (no está en el mapa actual)
`anger-volcano`	`AngerVolcanoGame`	Heredado
`guided-breathing`, `volcano-control`	2D	Heredados (respiración, volcán)
`fear-island`, `joy-valley`	2D	Heredados de la versión 2D

Identificador	Clase	Estado
`coming-soon`	`ComingSoonGame`	Marcador para islas sin contenido

Para poner un juego alternativo en el mapa basta con cambiar el campo `minigame` de la isla correspondiente en `src/data/islands.js`.

---

8. Datos: qué se guarda y dónde

El progreso del jugador no sale de su navegador. Todo vive en `localStorage`:

Clave	Contenido	Módulo
`emo-aventura-state`	Puntos, herramientas, insignias, actividades completadas, niveles por isla, intensidades, reevaluaciones, compromisos y ajustes (`sound`, `reduceMotion`)	`src/data/gameState.js`
`emotion-islands-player`	Perfil: nombre, avatar, color	`src/data/player.js`
`emotion-islands-progress`	Islas completadas (clave heredada, se fusiona con la anterior)	`src/ui/EmotionIslandApp.js`
`emoaventura.desagrado`	Respuestas de la Isla del Desagrado: por elemento (cueva, elemento, sentido, respuesta) y por ronda del espejo (señales elegidas, aciertos, confusiones)	`src/minigames/disgust/desagradoStore.js`

Borrar el progreso: basta con limpiar los datos del sitio en el navegador, o llamar a `resetGame()` desde `src/data/gameState.js`.

Exportar datos para investigación: hoy no hay exportación en la interfaz. Se puede leer desde la consola del navegador:

```
JSON.parse(localStorage.getItem('emo-aventura-state'));
JSON.parse(localStorage.getItem('emoaventura.desagrado'));
```

Los juegos autocontenidos (`valle-luz`, `valle-bruma`) dejan su telemetría en `sessionLog` / `sessionLogs` (consola).

Estadísticas anónimas de uso (Supabase)

Además de lo anterior, el juego envía **eventos anónimos** a Supabase para el panel de administración. Es **opcional**: si las variables de entorno están vacías, no se envía nada y el juego funciona igual.

Archivo	Papel
`.env` (a partir de `.env.example`)	`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_ADMIN_EMAIL`, `VITE_TRACK_DEV`. **No se sube al repositorio**

Archivo	Papel
`src/services/supabaseClient.js`	Carga la librería con `import()` dinámico; devuelve `null` si falta configuración
`src/services/tracking.js`	`trackEvent(isla, tipo, extra)`, cola de reintentos y reglas de apagado
`supabase/schema.sql`	Tablas, índices, políticas RLS y las cinco funciones `admin_*`
`src/admin/` + `admin.html`	El panel: página aparte, sin Three.js ni minijuegos

Tabla `island_events` (una fila por evento):

Columna	Valores
`island_id`	`sadness`, `anger`, `fear`, `disgust`, `frustration`, `calm`
`event_type`	`island_view`, `game_start`, `game_complete`, `game_exit`
`minigame`	identificador del minijuego (máximo 60 caracteres)
`session_id`	UUID aleatorio guardado en `sessionStorage`
`duration_ms`	0 a 14 400 000 (4 horas)
`device`	`mobile`, `tablet`, `desktop`
`created_at`	fecha y hora del servidor

Reglas que cumple el registro:

1. **Nunca bloquea el juego:** el envío no se espera (`await` fuera del flujo); cualquier fallo se ignora en silencio.
2. **Cola de reintentos:** hasta 50 eventos en `emoaventura.pendingEvents`; se vacían al volver la conexión (evento `online`).
3. **Sin datos personales:** no se envía perfil, puntos, intensidades ni respuestas abiertas.
4. **Apagado automático** si falta configuración, si la dirección lleva `?notrack=1` o si se está en desarrollo sin `VITE_TRACK_DEV=true`.
5. **Antirrebote** de 30 s para el mismo evento repetido y validación de las seis islas antes de enviar.
6. **Solo la clave** `anon` viaja al navegador; `service_role` no aparece en el código ni en los manuales.

Aparte de esto, `src/main.js` llama al contador público de visitas `abacus.jasoncameron.dev` en modo `no-cors`: solo suma una carga de página y no envía ni lee ningún dato. Se puede quitar borrando ese `fetch`.

La puesta en marcha (crear el proyecto, pegar el `schema.sql`, el correo con el código y el `.env`) está en Panel de administración · configuración paso a paso.

---

9. Editar contenido sin tocar la lógica

Los textos y el contenido pedagógico están **separados de la mecánica**. Para cambiar redacción, ejemplos o escenas, edita solo estos archivos:

Isla / zona	Archivo	Qué contiene
Desagrado	`src/minigames/disgust/DesagradoTextos.js`	Nombre de la isla, menú, las 4 cuevas y sus 24 elementos, las 8 señales de desagrado y los distractores con su explicación, las 3 escenas del espejo, los cierres
Herramientas e insignias	`src/data/tools.js`	Icono, nombre, emoción, estrategia y descripción de cada herramienta
Islas del mapa	`src/data/islands.js`	Nombre visible, subtítulo, emoji, paleta, posición y minijuego asociado
Resto de islas	Constantes al principio de cada `*Game.js`	Textos y parámetros de dificultad

Reglas al editar contenido (ver *Referencias y normas*):

- Tutear siempre al jugador; español neutro.
- Nada de gore, vómito explícito ni imágenes repulsivas: el desagrado se sugiere con caricatura e iconos.
- El rechazo se dirige siempre a una **conducta**, nunca a una persona, grupo, cultura, comida u origen.
- Ninguna respuesta del jugador puede restar puntos ni bloquear el avance.

---

10. Añadir una isla o un minijuego

1. **Crea la clase** en `src/minigames/<emocion>/MiJuego.js` extendiendo `MinigameBase`:

```
import { MinigameBase } from '../../engine/MinigameBase.js';

export class MiJuego extends MinigameBase {
  constructor(opts) { super({ ...opts, mode: 'third' }); }
  build() { /* escenario, HUD, entrada */ }
  onStart() { /* intro y primera consigna */ }
  onUpdate(dt) { /* bucle */ }
  onReset() { /* volver al estado inicial sin recargar */ }
  onDispose() { /* liberar lo propio */ }
  get completionPayload() {
    return { islandId: 'mi-isla', success: true, emoAventura: true, title: '…', message: '…' };
  }
}
```

1. **Abre el portal** al superar el reto (`this.openPortal(pos)`) y llama a `this.finish()` al cruzarlo, o llama directamente a `finish()` si la isla cierra con una tarjeta.
2. **Regístrala** en `src/minigames/index.js` con un identificador nuevo.
3. **Apunta la isla** en `src/data/islands.js` (`minigame: 'mi-identificador'`) y añade su etiqueta en `minigameLabels`.
4. **Recompensas:** añade la herramienta en `src/data/tools.js` y entrégala con `addReward('id-herramienta')`.

5. **Estilos propios:** crea `src/styles/mi-isla.css` e impórtalo en `src/main.js`.
6. **Comprueba la limpieza:** tras `dispose()` no deben quedar `canvas` huérfanos, temporizadores ni escuchadores (hay comprobaciones en las pruebas de §13).

---

11. Audio: cómo funciona y cómo añadir sonidos

`src/engine/AudioBus.js` sintetiza **todos** los efectos en tiempo de carga (`RECIPES`): ruido filtrado, tonos con armónicos, acordes y mezclas. No hay archivos de audio en el repositorio.

```
// añadir un sonido nuevo
miSonido: (ctx) => toneBuffer(ctx, 0.6, 440, 660, (p) => bell(p) * 0.4, 3),
// usarlo
this.audio.play('miSonido', { volume: 0.5 });
this.audio.playAt('miSonido', objeto3D, { volume: 0.8, refDistance: 6 });
```

- **Música generativa:** `CalmMusic`, `SadnessMusic` y `MysteryMusic` crean osciladores sobre el mismo `AudioContext`; exponen `start()`, `stop()`, `setVolume()`, `duck()` y un control de intensidad (`setLayers` / `setProgreso`).
- **Pasos:** cada isla puede ajustarlos con `this.stepVolume` y `this.stepSounds` (la Calma los deja casi mudos con `stepSoft`).
- **Silencio:** el botón actúa sobre el `master`; **todo** debe colgar de él (incluidos los sonidos posicionales) para que el silencio sea real.
- **Regla de diseño sonoro:** sin sonidos que sobresalten, sin agudos estridentes sostenidos y con entrada suave (*fade in*) en los sonidos largos.

---

12. Rendimiento y accesibilidad en el código

- **Objetivo:** 60 fps en gama media, nunca por debajo de 30. `MinigameBase` reduce el búfer interno antes que perder fluidez y lo recupera cuando sobra margen (`adaptQuality`).
- **Buenas prácticas ya aplicadas:** `InstancedMesh` para vegetación, *pooling* de partículas, materiales compartidos, geometrías reutilizadas y liberación explícita en `dispose()`.
- **Accesibilidad exigida a cada isla:**
  - botón de silencio siempre visible;
  - alternativa de **clic simple** allí donde haya arrastre;
  - texto con contraste ≥ 4.5:1 y tamaño mínimo legible (16 px en cuerpo);
  - `Reducir movimiento` respetado (`gameState.settings.reduceMotion` y `prefers-reduced-motion`);
  - nada que dependa del color como única señal;
  - carteles que no tapen elementos interactivos en pantallas bajas (ver `encuadrar()` y las anclas con zona segura en `desagradoUi.js`).

---

13. Pruebas y verificación

Comprobación rápida

```
corepack pnpm build          # debe terminar sin errores
node scripts/verify.mjs      # comprobaciones automáticas del proyecto
corepack pnpm fps            # medición de fotogramas
```

Pruebas de extremo a extremo (recomendado)

Se usan guiones de **Playwright sin cabeza** que sirven `/docs` interceptando las peticiones (sin abrir ningún puerto) y conducen el juego con gestos reales:

```
// patrón usado en los guiones de prueba
const browser = await chromium.launch({ headless: true, args: ['--use-gl=swiftshader'] });
await page.route('http://emo.test/**', servirDesdeDocs);
await page.evaluate(() => window.emoAventura.replay('disgust'));
// avanzar el juego de forma determinista:
await page.evaluate(() => { const g = window.emoAventura.currentMinigame.current; g.step(1 / 30);
});
```

Puntos a cubrir en cada isla antes de publicar:

- ☐ Se entra, se juega de principio a fin y se sale al mapa.
- ☐ Reiniciar desde la pausa deja el estado limpio.
- ☐ Salir al mapa libera todo (sin `canvas` huérfanos).
- ☐ Sin errores en la consola del navegador.
- ☐ En teléfono **tumbado (≈800×360)** y **de pie (≈390×780)**: nada tapado, todo alcanzable, botones de cierre visibles.
- ☐ El progreso queda guardado en `localStorage` con los valores esperados.

---

14. Control de versiones y flujo de trabajo

- Rama principal: **`main`** (es la que publica GitHub Pages).
- Ramas de trabajo: `checkup/*`, `fix/*`, `feat/*`. **Comprueba en qué rama estás antes de commitear**: `git status -sb`.
- Para llevar una rama a producción, avance recto:

```
git checkout main
git merge --ff-only mi-rama
git push origin main
```

- Cada cambio de código publicado debe ir acompañado de su `/docs` recompilado.
- Mensajes de commit en español, describiendo el efecto para quien juega.

---

15. Resolución de problemas

Síntoma	Causa probable	Solución
Pantalla en blanco al abrir	`localStorage` con progreso corrupto de una versión anterior	El arranque ya lo detecta y reinicia; si persiste, borra los datos del sitio
La página muestra una versión vieja	Caché del navegador	El sello `__BUILD__` fuerza recarga; si no, `Ctrl+F5`
`pnpm: command not found`	pnpm no está en el PATH	Usar `corepack pnpm …`
El build no cambia nada visible	No se recompiló `/docs` o no se publicó	`corepack pnpm build` y commitear `/docs`
Va a tirones	GPU limitada o muchas pestañas	Activar *Reducir movimiento*, cerrar pestañas; `F3` para medir
No suena nada	El navegador exige un gesto previo para el audio	Tocar la pantalla; comprobar el botón de sonido
Un juego se queda trabado tras ESC	Estado de pausa mal restaurado	Ya corregido en el motor; reiniciar desde la pausa
`/docs` siempre aparece modificado en git	Sello de versión por diseño	Si solo compilaste para probar: `git checkout -- docs`

16. Mantenimiento y hoja de ruta

Tareas periódicas

- Actualizar dependencias (`three`, `vite`) y volver a pasar las pruebas de §13.
- Revisar que la URL pública carga y que Pages sigue apuntando a `main` + `/docs`.
- Revisar textos y ejemplos con el equipo pedagógico al menos una vez por curso.

Deuda técnica conocida

- Los *assets* de la Isla del Desagrado son **marcadores generados por código** (`PLACEHOLDER_*` en `desagradoAssets.js`) y los seis sonidos de la caverna son sintetizados (`PLACEHOLDER_AUDIO_*`): conviene sustituirlos por sprites, modelos y grabaciones reales.
- Quedan minijuegos heredados registrados pero fuera del mapa (§7): decidir si se retiran o se reincorporan.
- No hay exportación de datos desde la interfaz (§8), útil si se quiere usar el juego en un estudio.